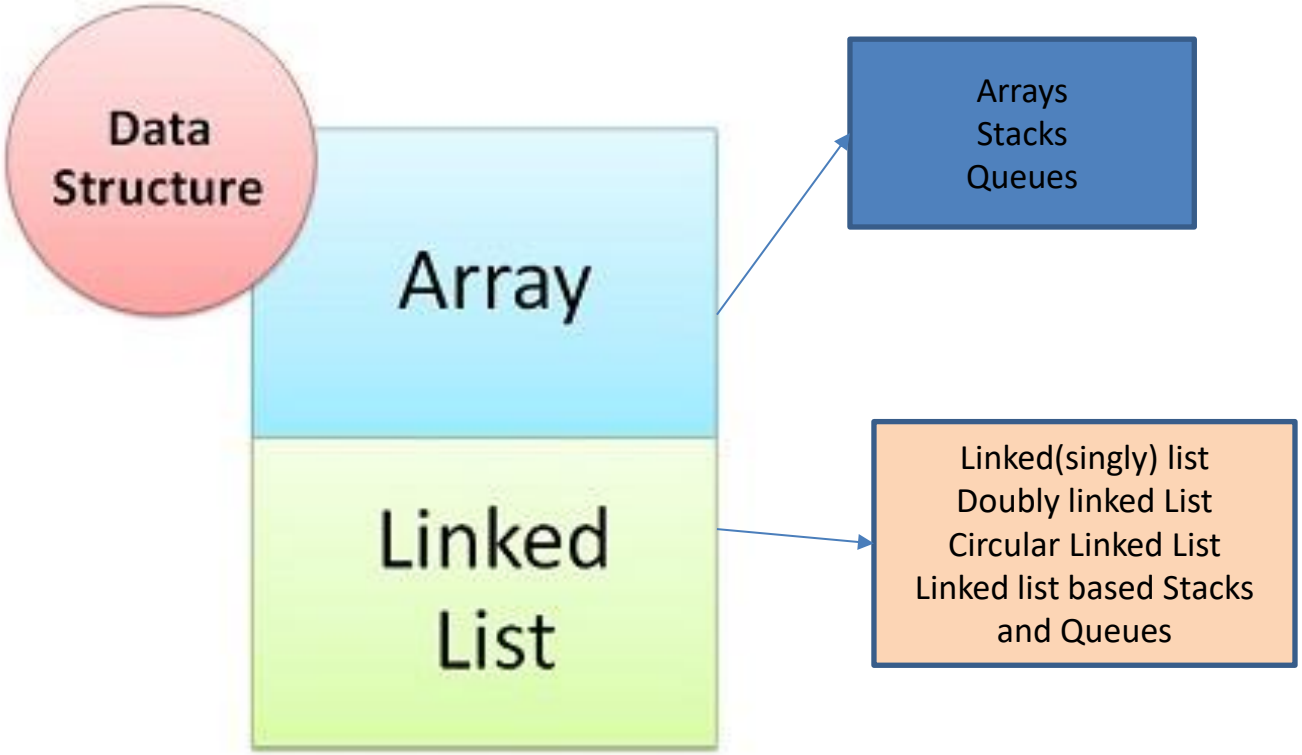
# Data Structures and Algorithms
# Lecture 4 – Linked List

**M.G.Noel A.S Fernando (PhD)**

**Professor**

**UCSC**

# List Data Structures



# Array based and Linked List structures

- The array is a linear collection of data elements in which the elements are stored

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| 40 | 55 | 63 | 17 | 22 | 68 | 89 | 97 | 89 |
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |

<- **Array Indices**

**Array Length = 9**
**First Index = 0**
**Last Index = 8**

# Array based and Linked List structures

- The array is a linear collection of data elements in which the elements are stored in consecutive memory locations.

- While declaring arrays, we have to specify the size of the array, which will restrict the number of elements that the array can store.
- For example, if we declare an array as int marks[10], then the array can store a maximum of 10 data elements but not more than that.

# Advantages

- No need to declare large number of variables individually.
- Variables are not scattered in memory , they are stored in contiguous memory.
- Ease the handling of large no of variables of same datatype.

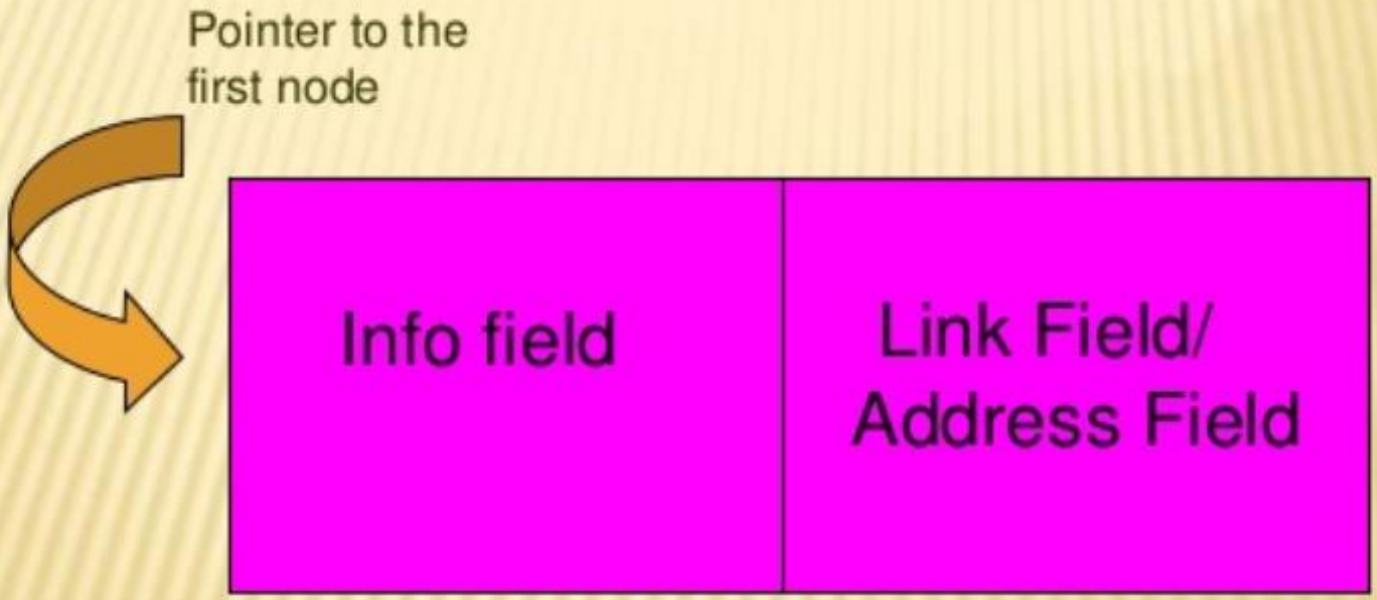
# LIMITATION

- Rigid structure.
- Can be hard to add/remove elements.
- Cannot be dynamically resized in most languages.
- Memory loss.

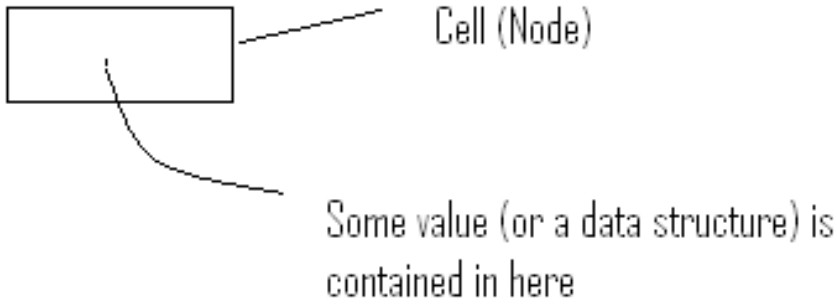
# LINKED LISTS

# What is Linked List?

A linked list is a collection of nodes with various fields
It contains data field and Address field or Link field

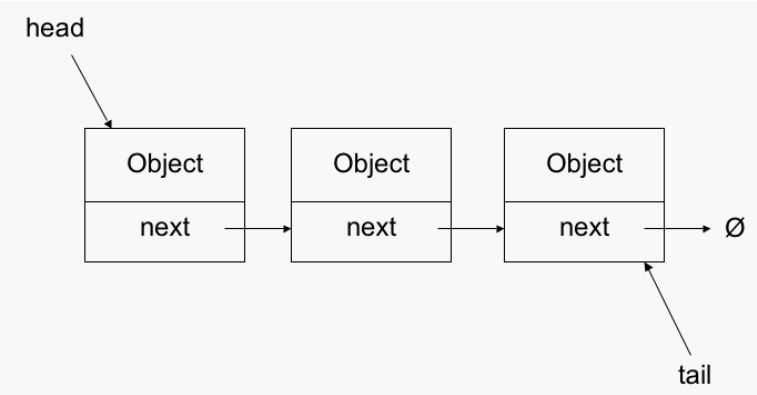


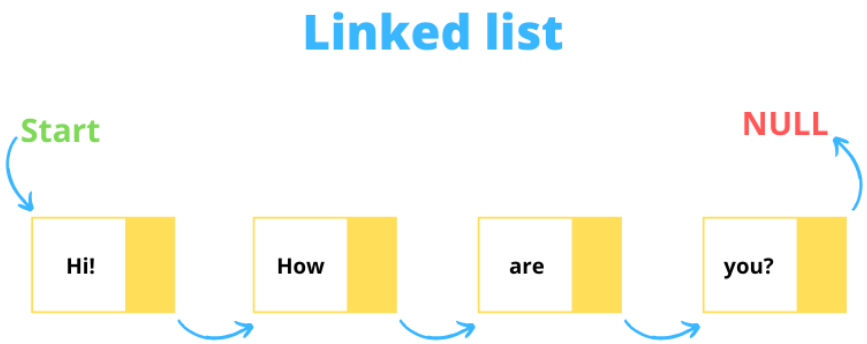
# Example




- Cells (nodes) like these are connected in various ways to build a data structure.

# Linked List as a Data Structure

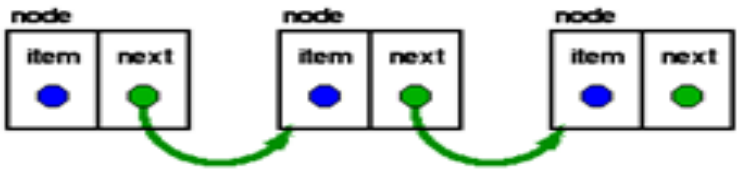


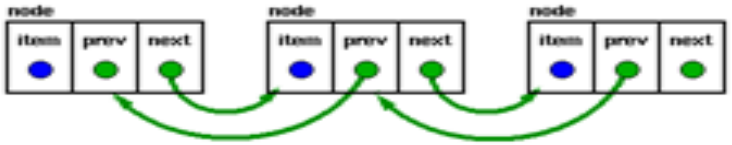
# Linked lists

## Lists

- A list




- A list where we can navigate backwards as well

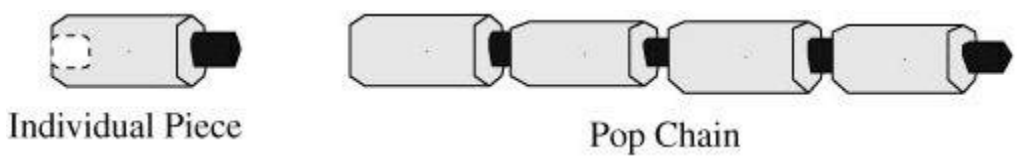


# What is a Linked List?

- Each element (node) inside a linked list is linked to the previous node and successor (next) node.
- This allows for more efficient insertion and deletion of nodes.

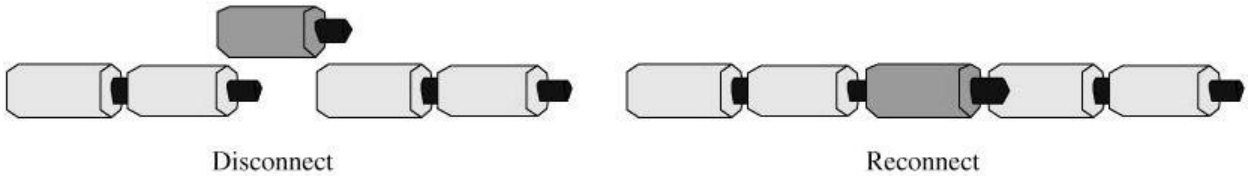
# What is linked list ?(Continued)

- Think of each element in a linked list as being an individual piece in a child's pop chain. To form a chain, insert the connector into the back of the next piece



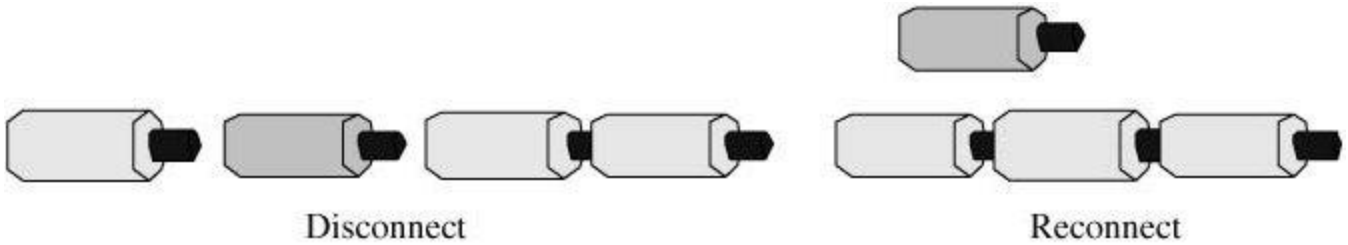
# Introducing Linked Lists (continued)

- Inserting a new piece into the chain involves merely breaking a connection and reconnecting the chain at both ends of the new piece.



# What is linked list(continued)

- Removal of a piece from anywhere in the chain requires breaking its two connections, removing the piece, and then reconnecting the chain.



# Details definition of Linked List

- Each item has a data part (one or more data members), and a link that points to the next item.
- One natural way to implement the link is as a pointer; that is, the link is the address of the next item in the list.
- It makes good sense to view each item as an object, that is, as an instance of a class.
- We call that class: Node
- The last item does not point to anything. We set its link member to **NULL**. This is denoted graphically by a self-loop

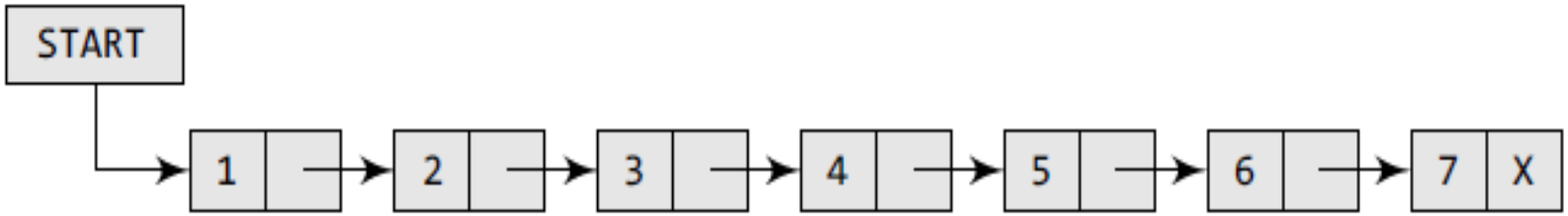
# Array Vs Linked Lists

| Arrays | Linked List |
|---|---|
| Fixed Size | Dynamic Size |
| Insertions and deletions are inefficient :elements are usually shifted | Insertions and deletions are efficient :no shifting |
| Random access : efficient indexing | No random access |
| No memory wastage, if array is full or almost full. Otherwise, memory wastage | Since memory is dynamically allocated (according to our requirement) there is no waste of memory |
| Sequential access is faster [Reason : Elements in contiguous memory locations] | Sequential access is not faster [Reason : Elements not in contiguous memory locations] |

# Basic Terminologies

- A linked list, is a linear collection of data elements.
- These data elements are called *nodes*.
- It acts as a building block to implement data structures such as stacks, queues, and their variations.
- A linked list can be perceived as a train or a sequence of nodes in which each node contains one or more data fields and a pointer to the next node.

# Simple linked list

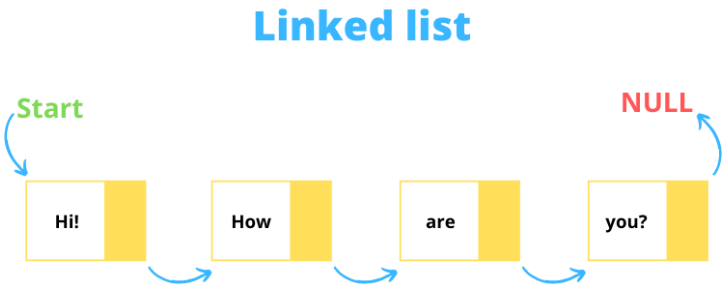



Linked lists contain a pointer variable `START` that stores the address of the first node in the list.

We can traverse the entire list using `START` which contains the address of the first node;
If `START = NULL`, then the linked list is empty and contains no nodes

# In C, we can implement a linked list using the following code:

```
struct node
{
        int data;
        struct node *next;
};
```




Note :This is denoted graphically a self loop

# Dynamic memory allocation

- *Dynamic memory* = memory allocated at run-time.
- *Static memory* = memory allocated at compile-time.
- Programming languages like C and Pascal provide libraries for managing dynamic memory.
- This memory is allocated from a special area called *heap*.
- The heap is managed by a special language component that implements functions like new and dispose in Pascal or malloc(), calloc() and free() in C.

# How a linked list is maintained in the memory

- The *node* which has two fields, DATA and NEXT.
- DATA will store in the information part and NEXT will store the address of the next node in sequence.
- The variable START is used to store the address of the first node.
- Here, in this example, START = 1, so the first data is stored at address 1, which is H.

# How a linked list is maintained in the memory

START

1

| | Data | Next |
|---|---|---|
| 1 | H | 4 |
| 2 | | |
| 3 | | |
| 4 | E | 7 |
| 5 | | |
| 6 | | |
| 7 | L | 8 |
| 8 | L | 10 |
| 9 | | |
| 10 | O | −1 |

At the end, `NEXT` entry contains –1 or `NULL`

# Two different linked lists are simultaneously maintained in the memory

START 1

1 (Biology)

2

START 2
(Computer Science)

| | Roll No | Next |
|---|---|---|
| 1 | S01 | 3 |
| 2 | S02 | 5 |
| 3 | S03 | 8 |
| 4 | | |
| 5 | S04 | 7 |
| 6 | | |
| 7 | S05 | 10 |
| 8 | S06 | 11 |
| 9 | | |
| 10 | S07 | 12 |
| 11 | S08 | 13 |
| 12 | S09 | -1 |
| 13 | S10 | 15 |
| 14 | | |
| 15 | S11 | -1 |

# Homework

- **Discuss the advantages, disadvantages, similarities and differences of Linked Lists versus Arrays.**
- **You may use the following table to answer the question**

# Sample Students' linked list

START
18

| | Roll No | Name | Aggregate | Grade | Next |
|---|---|---|---|---|---|
| 1 | S01 | Ram | 78 | Distinction | 6 |
| 2 | S02 | Shyam | 64 | First division | 14 |
| 3 | | | | | |
| 4 | S03 | Mohit | 89 | Outstanding | 17 |
| 5 | | | | | |
| 6 | S04 | Rohit | 77 | Distinction | 2 |
| 7 | S05 | Varun | 86 | Outstanding | 10 |
| 8 | S06 | Karan | 65 | First division | 12 |
| 9 | | | | | |
| 10 | S07 | Veena | 54 | Second division | -1 |
| 11 | S08 | Meera | 67 | First division | 4 |
| 12 | S09 | Krish | 45 | Third division | 13 |
| 13 | S10 | Kusum | 91 | Outstanding | 11 |
| 14 | S11 | Silky | 72 | First division | 7 |
| 15 | | | | | |
| 16 | | | | | |
| 17 | S12 | Monica | 75 | Distinction | 1 |
| 18 | S13 | Ashish | 63 | First division | 19 |
| 19 | S14 | Gaurav | 61 | First division | 8 |

# Memory management in C

- The C programming language provides several functions for memory allocation and management.
- These functions can be found in the **<stdlib.h>** header file.
- Although C inherently does not have any techniques to allocate memory dynamically. There are 4 library functions defined under <stdlib.h> for dynamic memory allocation.

# Memory management functions in C

| Function | Use of Function |
| --- | --- |
| malloc() | Allocates requested size of bytes and returns a pointer first byte of allocated space |
| calloc() | Allocates space for an array elements, initializes to zero and then returns a pointer to memory |
| free() | deallocate the previously allocated space |
| realloc() | Change the size of previously allocated space |

# C malloc()

- The name malloc stands for “memory allocation”
- The function malloc() reserves a block of memory of specified size and return a pointer of type void which can be casted into pointer of any form.
- Syntax

Ptr = (cast-type*)  malloc (byte-size)

- Here, ptr is pointer of cast-type. The malloc() function returns a pointer to an area of memory with size of byte size.
- If the space is insufficient, allocation fails and returns NULL pointer.

# C malloc()

```
Ptr= (int*) malloc (100*sizeof(int);
```

- This statement will allocate either 200 or 400 according to size int 2 or 4 bytes respectively and the pointer points to the first byte of the memory.

# C - calloc() – stands for “contiguous allocation”

- **Syntax of calloc()**

Ptr = (cast-type*) calloc (n,element-size);

- This statement will allocate contiguous space in memory for an array of n elements.
- Example
- Ptr=(float*)calloc(25,sizeof(float));
- This statement allocates contiguous space in memory for an array of 25 elements each of size of float, i.e, 4 bytes

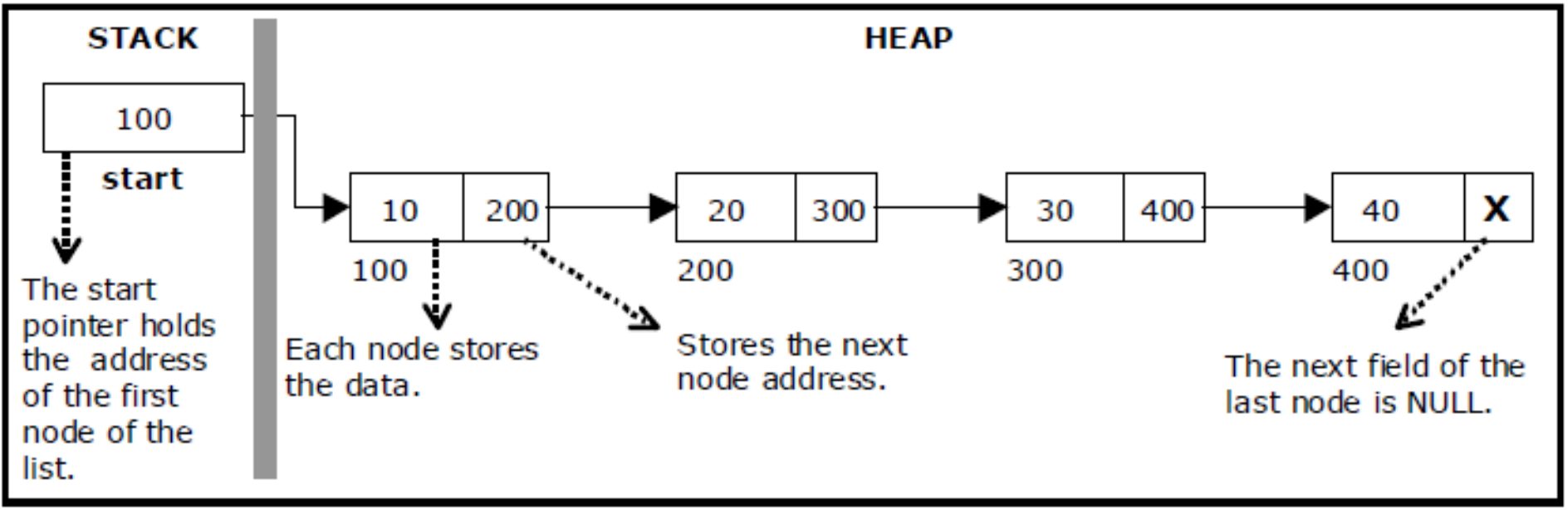
# C - free()

- Dynamically allocated memory created with either calloc() or malloc() doesn't get freed on its own. You must explicitly use free() to release the space.
- **syntax of free()**
- Free(ptr);
- This statement frees (release) the space allocated in the memory in the memory pointed ptr.

# Single Linked List

- A linked list allocates space for each element separately in its own block of memory called a "node".
- The list gets an overall structure by using pointers to connect all its nodes together like the links in a chain. Each node contains two fields; a "data" field to store whatever element, and a "next" field which is a pointer used to link to the next node.
- Each node is allocated in the heap using malloc(), so the node memory continues to exist until it is explicitly de-allocated using free().
- The front of the list is a pointer to the “start” node.

# Example :A single linked list



# Implementation of Single Linked List

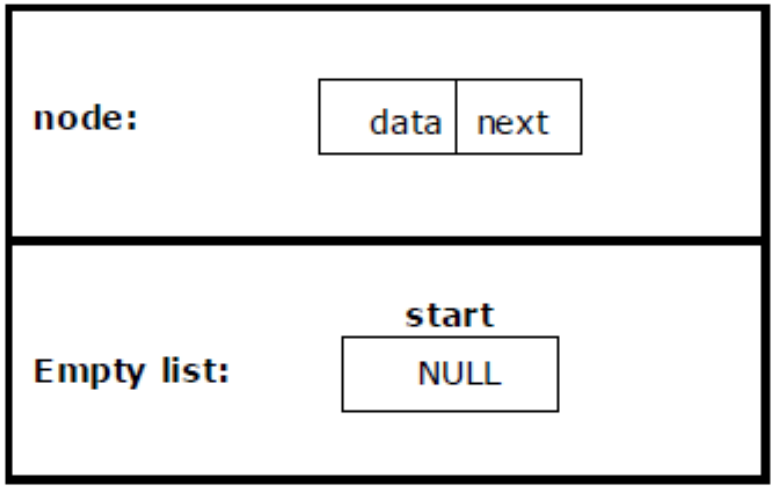
- Before writing the code to build the above list, we need to create a **start** node, used to create and access other nodes in the linked list.
- Structure definition, single link node and empty list:
- Creating a structure with one data item and a next pointer, which will be pointing to next node of the list.
- This is called as self-referential structure.
- Initialize the start pointer to be NULL.

# Structure definition, single link node and empty list

```
struct slinklist
{
        int data;
        struct slinklist* next;
};

typedef struct slinklist node;

node *start = NULL;
```



# The basic operations in a single linked list are:

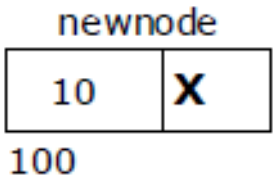
- Creation.
- Insertion.
- Deletion.
- Traversing

# Creating a node for Single Linked List:

- Creating a singly linked list starts with creating a node.
- Sufficient memory has to be allocated for creating a node.
- The information is stored in the memory, allocated by using the malloc() function.
- The function getnode(), is used for creating a node, after allocating memory for the structure of type node, the information for the item (i.e., data) has to be read from the user, set next field to NULL and finally returns the address of the node. The following Figure illustrates the creation of a node for single linked list.

# The following figure illustrates the creation of a node for single linked list.

```
node* getnode()
{
    node* newnode;
    newnode = (node *) malloc(sizeof(node));
    printf("\n Enter data: ");
    scanf("%d", &newnode -> data);
    newnode -> next = NULL;
    return newnode;
}
```

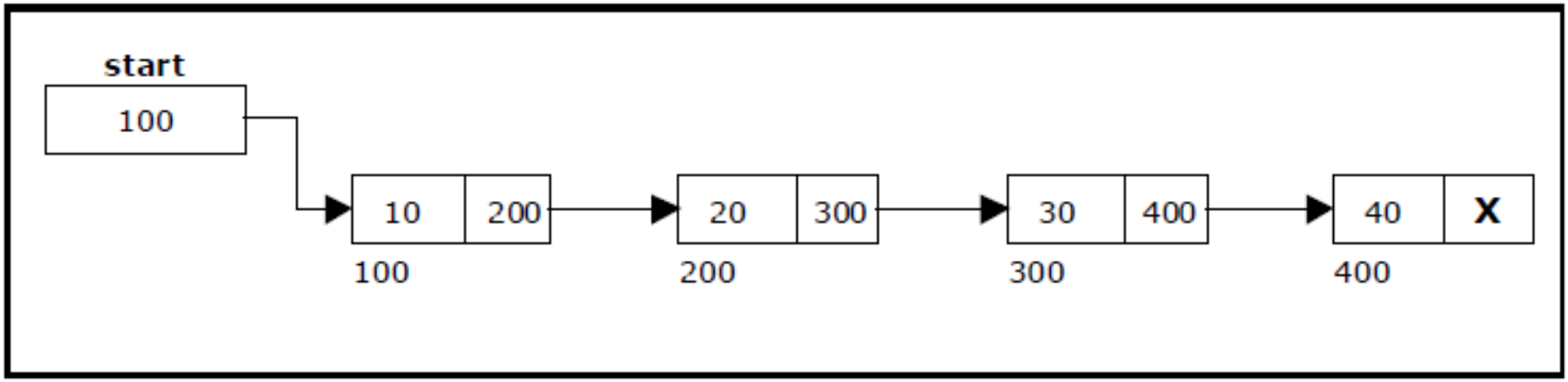



newnode with a value of 10

# Creating a Singly Linked List with ‘n’ number of nodes:

- The following steps are to be followed to create ‘n’ number of nodes:
  - Get the new node using getnode().
    newnode = getnode();
  - If the list is empty, assign new node as start.
    start = newnode;
  - If the list is not empty, follow the steps given below:
    - The next field of the new node is made to point the first node (i.e. start node) in the list by assigning the address of the first node.
    - The start pointer is made to point the new node by assigning the address of the new node.
  - Repeat the above steps ‘n’ times.

# 4 items in a single linked list stored at different locations in memory.



The function createlist(), is used to create ‘n’ number of nodes:

```
void createlist(int n)
{
        int i;
        node *newnode;
        node *temp;
        for(i = 0; i < n ; i++)
        {
                newnode = getnode();
                if(start == NULL)
                {
                        start = newnode;
                }
                else
                {
                        temp = start;
                        while(temp -> next != NULL)
                                temp = temp -> next;
                        temp -> next = newnode;
                }
        }
}
```

# Traversing a Linked List

- Traversing a linked list means accessing the nodes of the list in order to perform some processing on them.
- A linked list always contains a pointer variable START which stores the address of the first node of the list.
- End of the list is marked by storing NULL in the NEXT field of the last node.
- For traversing the linked list, we also make use of another pointer variable PTR which points to the node that is currently being accessed.

# Algorithm for traversing a linked list

```
Step 1: [INITIALIZE] SET PTR = START
Step 2: Repeat Steps 3 and 4 while PTR != NULL
Step 3:            Apply Process to PTR -> DATA
Step 4:            SET PTR = PTR -> NEXT
       [END OF LOOP]
Step 5: EXIT
```

# Home work -

- write an pseudo code algorithm and C Program to count the number of nodes in a linked list.

# An algorithm to count the number of nodes in a linked list

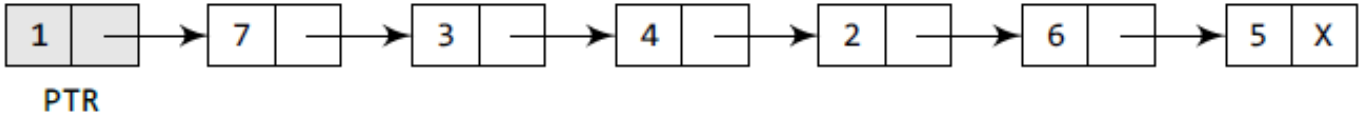
```
Step 1: [INITIALIZE] SET COUNT = 0
Step 2: [INITIALIZE] SET PTR = START
Step 3: Repeat Steps 4 and 5 while PTR != NULL
Step 4:              SET COUNT = COUNT + 1
Step 5:              SET PTR = PTR -> NEXT
        [END OF LOOP]
Step 6: Write COUNT
Step 7: EXIT
```

# Searching for a Value in a Linked List

- A linked list consists of nodes which are divided into two parts, the information part and the next part.
- Searching means finding whether a given value is present in the information part of the node or not.
- If it is present, the algorithm returns the address of the node that contains the value.

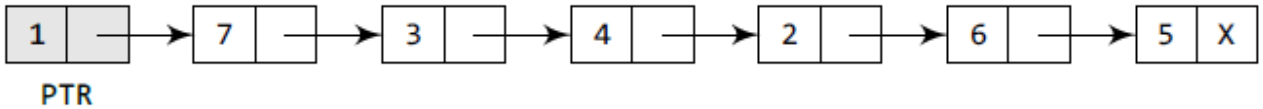
# Example - **Searching for a Value in a Linked List**

- Consider the following linked list.



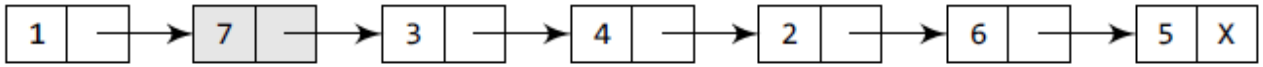

- If we wants to search a node with VAL = 4, then the flow of the algorithm can be represented as follows:

# Example - **Searching for a Value in a Linked List – cont….**



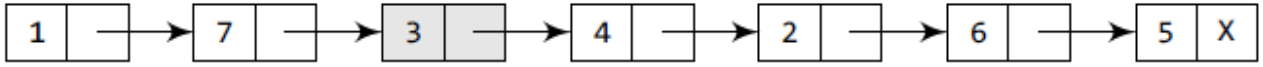

PTR

Here PTR –> DATA = 1. Since PTR –> DATA != 4, we move to the next node.



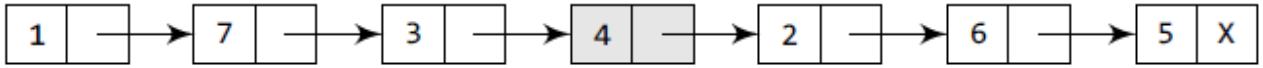

PTR

Here PTR –> DATA = 7. Since PTR –> DATA != 4, we move to the next node.




PTR

Here PTR –> DATA = 3. Since PTR –> DATA != 4, we move to the next node.




PTR

Here PTR –> DATA = 4. Since PTR –> DATA = 4, POS = PTR. POS now stores the address of the node that contains VAL

# Algorithm to search a node from a linked list

```
Step 1: [INITIALIZE] SET PTR = START
Step 2: Repeat Step 3 while PTR != NULL
Step 3:     IF VAL = PTR -> DATA
                SET POS = PTR
                Go To Step 5
            ELSE
                SET PTR = PTR -> NEXT
            [END OF IF]
        [END OF LOOP]
Step 4: SET POS = NULL
Step 5: EXIT
```

# Homework :Question 1

- **Write a C program to find sum of n elements entered by user.**
- **Hint :To perform this program, allocate memory dynamically using malloc() function.**
- **Answer**
- **https://www.programiz.com/c-programming/c-dynamic-memory-allocation**

# Answer

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
     int num, i, *ptr, sum = 0;

     printf("Enter number of elements: ");
     scanf("%d", &num);

     ptr = (int*) malloc(num * sizeof(int));   //memory allocated using malloc
     if(ptr == NULL)
     {
          printf("Error! memory not allocated.");
          exit(0);
     }

     printf("Enter elements of array: ");
     for(i = 0; i < num; ++i)
     {
          scanf("%d", ptr + i);
          sum += *(ptr + i);
     }

     printf("Sum = %d", sum);
     free(ptr);
     return 0;
}
```

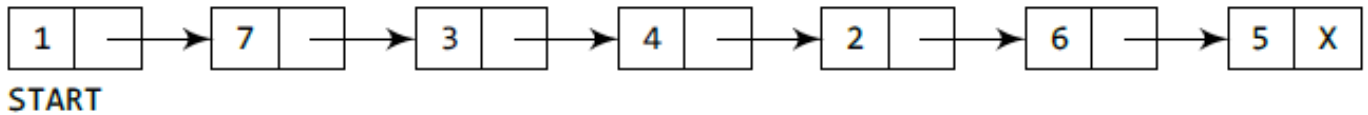
# Inserting a New Node in a Linked List

We will consider  four insertion cases and then see how insertion is done in each case.

- Case 1: The new node is inserted at the beginning.
- Case 2: The new node is inserted at the end.
- Case 3: The new node is inserted after a given node.
- Case 4: The new node is inserted before a given node.

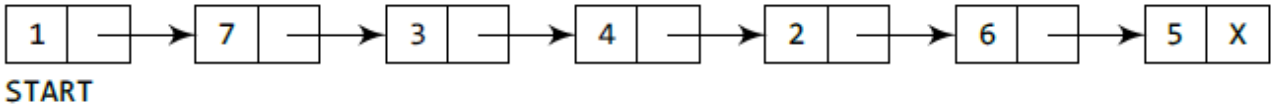
# Case 1: ***Inserting a Node at the Beginning of a Linked List***

- Note : In all these four cases, let us first discuss an important term called OVERFLOW.
- Overflow is a condition that occurs when AVAIL = NULL or no free memory cell is present in the system. When this condition occurs, the program must give an appropriate message.
- Assume that a node to be inserted to the following

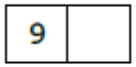



- A new node with data part 9 to be inserted to the beginning of the list.

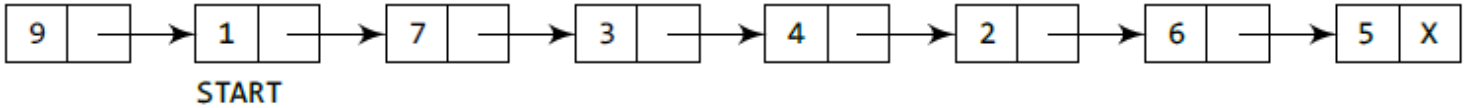
# Case 1: ***Inserting a Node at the Beginning of a Linked List cont.....***



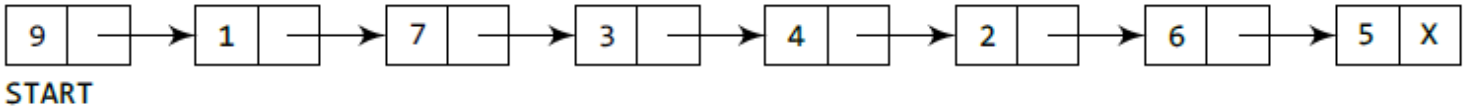

Allocate memory for the new node and initialize its DATA part to 9.




Add the new node as the first node of the list by making the NEXT part of the new node contain the address of START.




Now make START to point to the first node of the list.



# Case 1: ***Inserting a Node at the Beginning of a Linked List cont.....***

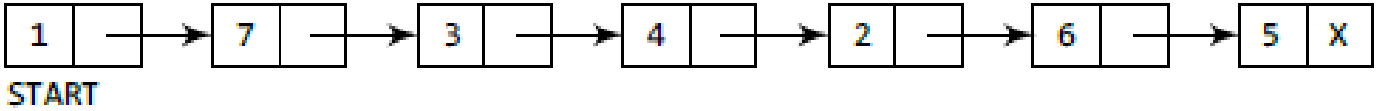
```
Step 1: IF AVAIL = NULL
            Write OVERFLOW
            Go to Step 7
       [END OF IF]
Step 2: SET NEW_NODE = AVAIL
Step 3: SET AVAIL = AVAIL -> NEXT
Step 4: SET NEW_NODE -> DATA = VAL
Step 5: SET NEW_NODE -> NEXT = START
Step 6: SET START = NEW_NODE
Step 7: EXIT
```

AVAIL is advanced

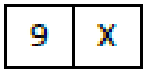
# *Case 2: Inserting a Node at the End of a Linked List*

- Suppose we want to add a new node with data 9 as the last node of the list.
- Then the following changes will be done in the linked list.
- we take a pointer variable PTR and initialize it with START. That is, PTR now points to the first node of the linked list.
- Using a while loop, we traverse through the linked list to reach the last node. Once we reach the last node, we change the NEXT pointer of the last node to store the address of the new node.
- the NEXT field of the new node contains NULL,which signifies the end of the linked list

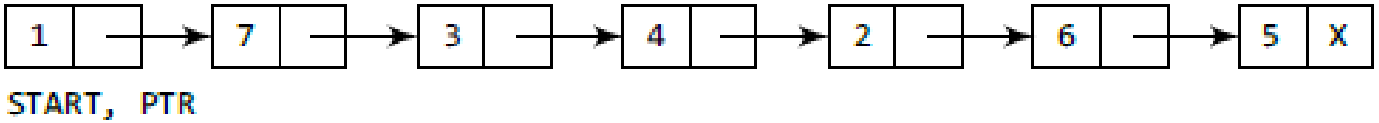
# *Case 2: Inserting a Node at the End of a Linked List: cont..*



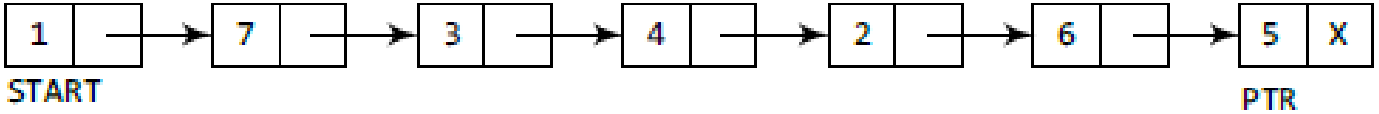

Allocate memory for the new node and initialize its DATA part to 9 and NEXT part to NULL.



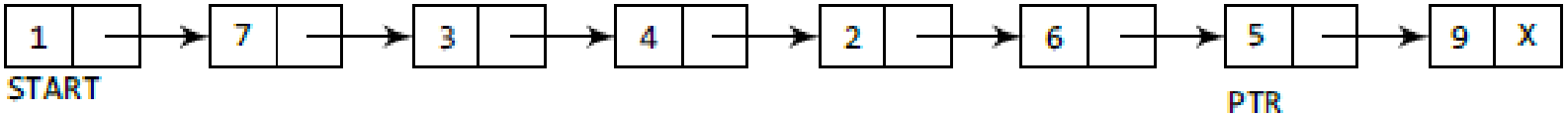

Take a pointer variable PTR which points to START.




Move PTR so that it points to the last node of the list.




Add the new node after the node pointed by PTR. This is done by storing the address of the new node in the NEXT part of PTR.

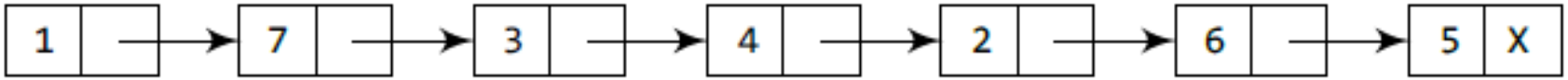


# *Case 2: Inserting a Node at the End of a Linked List: cont..*

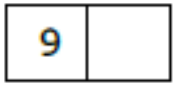
```
Step 1: IF AVAIL = NULL
            Write OVERFLOW
            Go to Step 10
       [END OF IF]
Step 2: SET NEW_NODE = AVAIL
Step 3: SET AVAIL = AVAIL->NEXT
Step 4: SET NEW_NODE->DATA = VAL
Step 5: SET NEW_NODE->NEXT = NULL
Step 6: SET PTR = START
Step 7: Repeat Step 8 while PTR->NEXT != NULL
Step 8:     SET PTR = PTR->NEXT
       [END OF LOOP]
Step 9: SET PTR->NEXT = NEW_NODE
Step 10: EXIT
```

# *Case 3: Inserting a Node After a Given Node in a Linked List*

- Suppose we want to add a new node with value 9 after the node containing data 3.

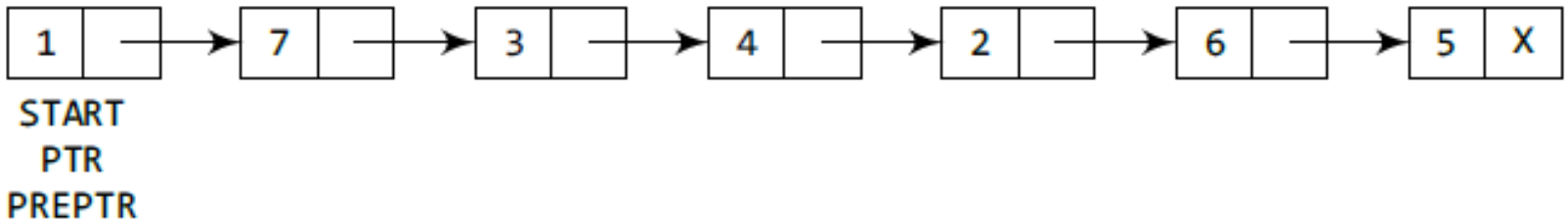



START
Allocate memory for the new node and initialize its DATA part to 9.

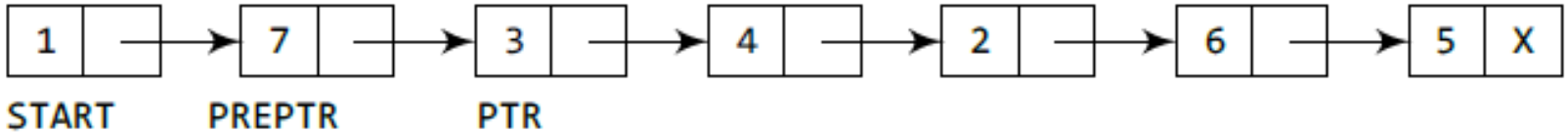


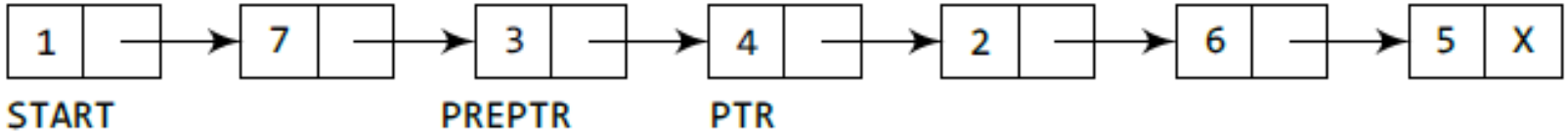

Take two pointer variables PTR and PREPTR and initialize them with START so that START, PTR, and PREPTR point to the first node of the list.

# *Case 3: Inserting a Node After a Given Node in a Linked List*



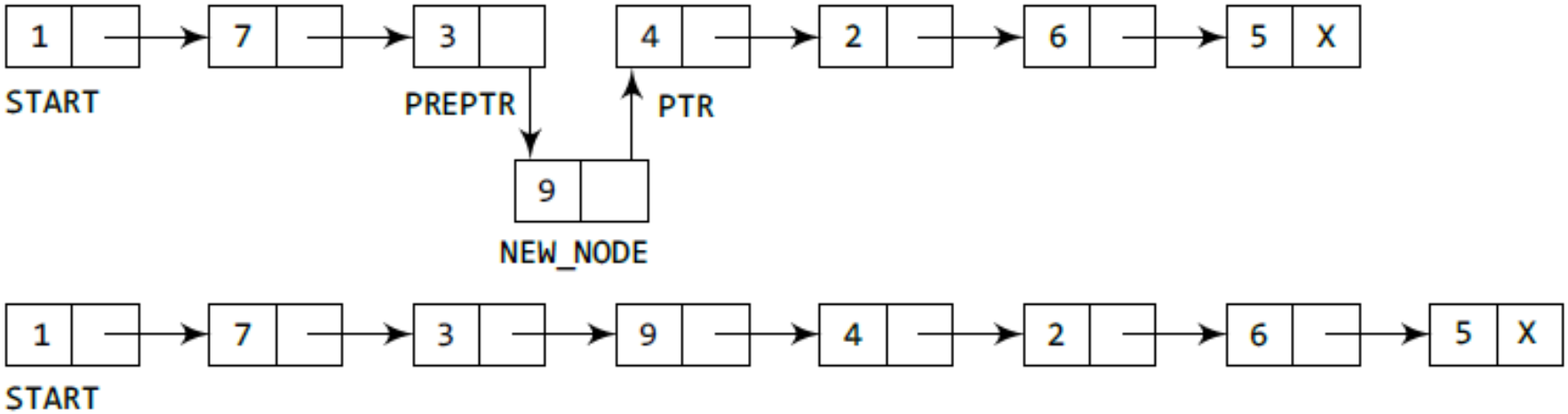

Move PTR and PREPTR until the DATA part of PREPTR = value of the node after which insertion has to be done. PREPTR will always point to the node just before PTR.



# *Case 3: Inserting a Node After a Given Node in a Linked List*

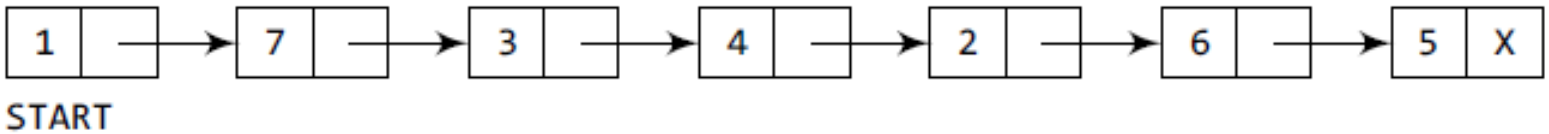
Add the new node in between the nodes pointed by PREPTR and PTR.



# *Case 3: Inserting a Node After a Given Node in a Linked List cont.....*

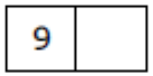
```
Step 1: IF AVAIL = NULL
            Write OVERFLOW
            Go to Step 12
       [END OF IF]
Step 2: SET NEW_NODE = AVAIL
Step 3: SET AVAIL = AVAIL->NEXT
Step 4: SET NEW_NODE->DATA = VAL
Step 5: SET PTR = START
Step 6: SET PREPTR = PTR
Step 7: Repeat Steps 8 and 9 while PREPTR->DATA
        != NUM
Step 8:     SET PREPTR = PTR
Step 9:     SET PTR = PTR->NEXT
         [END OF LOOP]
Step 10: PREPTR->NEXT = NEW_NODE
Step 11: SET NEW_NODE->NEXT = PTR
Step 12: EXIT
```

# *Case 4: Inserting a Node Before a Given Node in a Linked List*

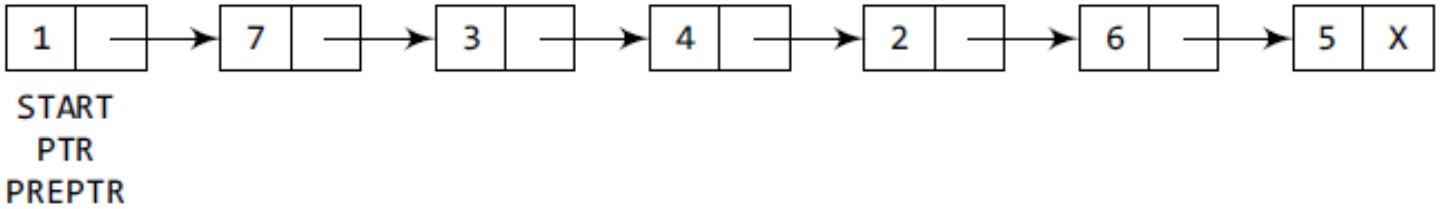



- Suppose we want to add a new node with value 9 before the node containing 3.

Allocate memory for the new node and initialize its DATA part to 9.

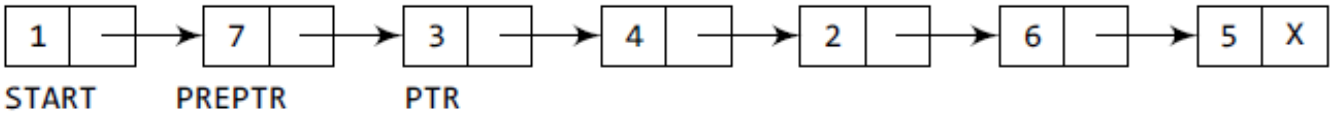



Initialize PREPTR and PTR to the START node.

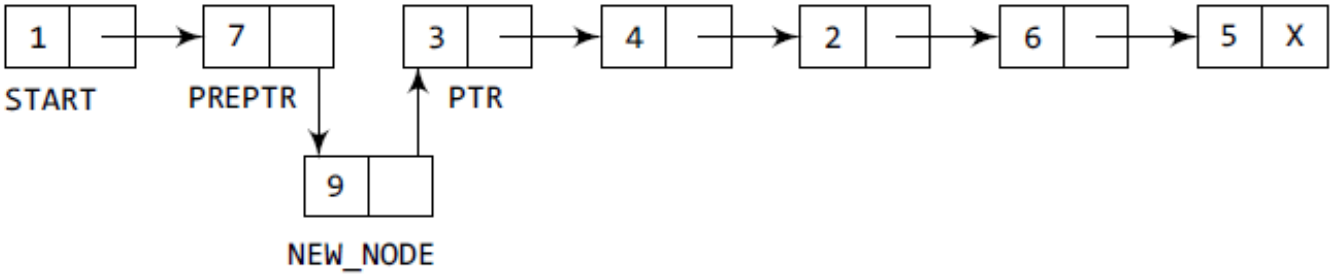


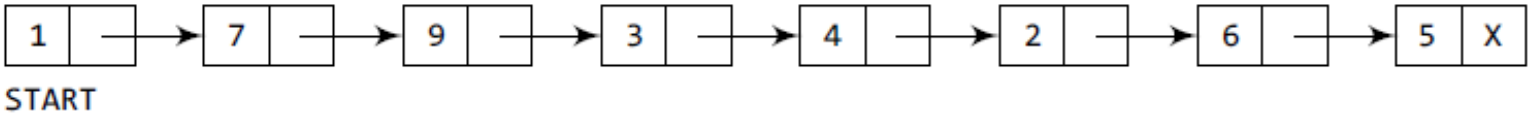
# *Inserting a Node Before a Given Node in a Linked List*

Move PTR and PREPTR until the DATA part of PTR = value of the node before which insertion has to be done. PREPTR will always point to the node just before PTR.




Insert the new node in between the nodes pointed by PREPTR and PTR.



# Algorithm to insert a new node before a node that has value NUM

```
Step 1: IF AVAIL = NULL
            Write OVERFLOW
            Go to Step 12
       [END OF IF]
Step 2: SET NEW_NODE = AVAIL
Step 3: SET AVAIL = AVAIL->NEXT
Step 4: SET NEW_NODE->DATA = VAL
Step 5: SET PTR = START
Step 6: SET PREPTR = PTR
Step 7: Repeat Steps 8 and 9 while PTR->DATA != NUM
Step 8:     SET PREPTR = PTR
Step 9:     SET PTR = PTR->NEXT
        [END OF LOOP]
Step 10: PREPTR->NEXT = NEW_NODE
Step 11: SET NEW_NODE->NEXT = PTR
Step 12: EXIT
```